

Teaching Exam

**Subject –
Computer
Science**

**Operating
System**



Lecture No –5

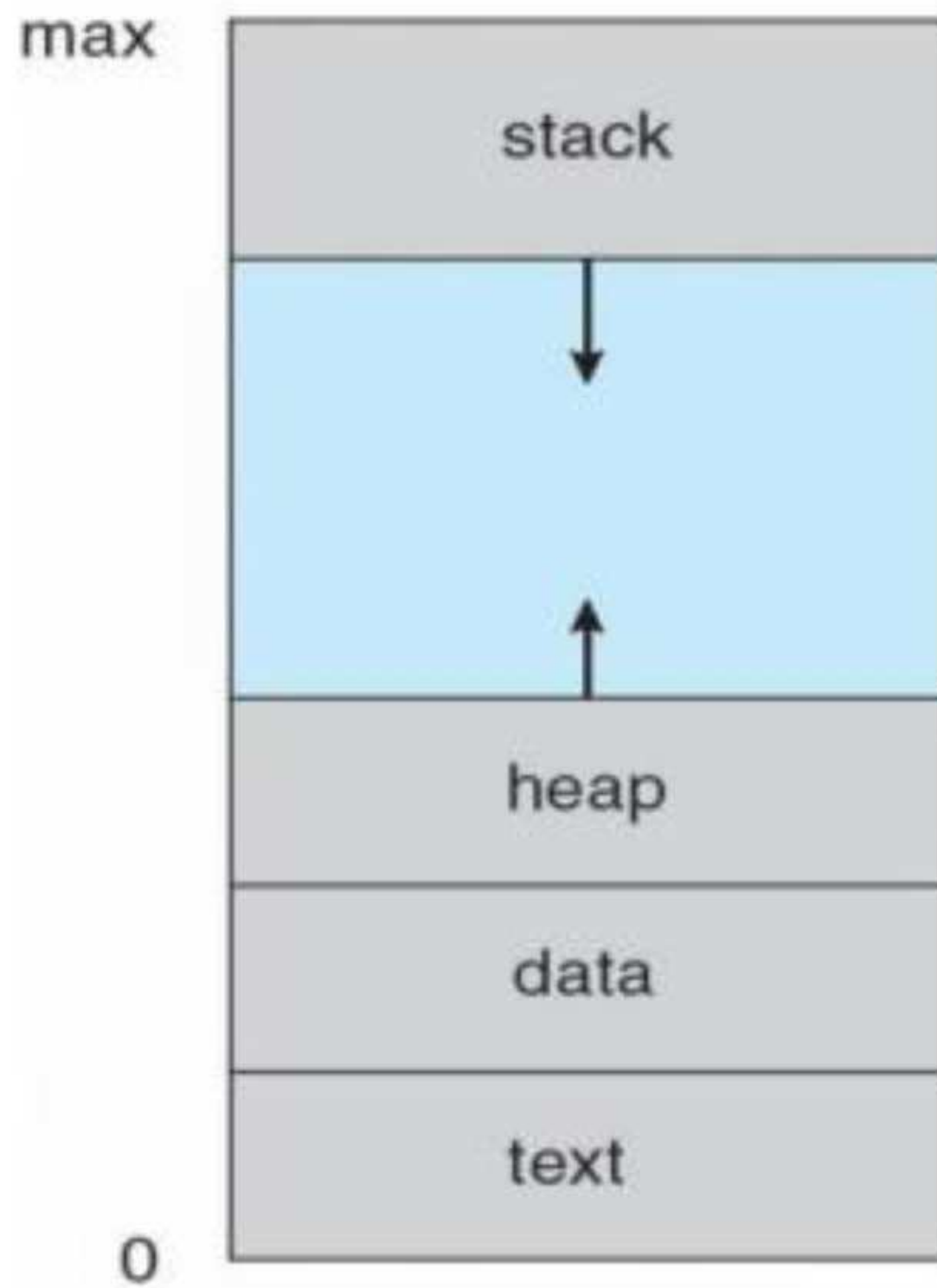
By–Satyendra Chaudhary Sir

Topics to be Covered

Topic

Process synchronization
CPU scheduling



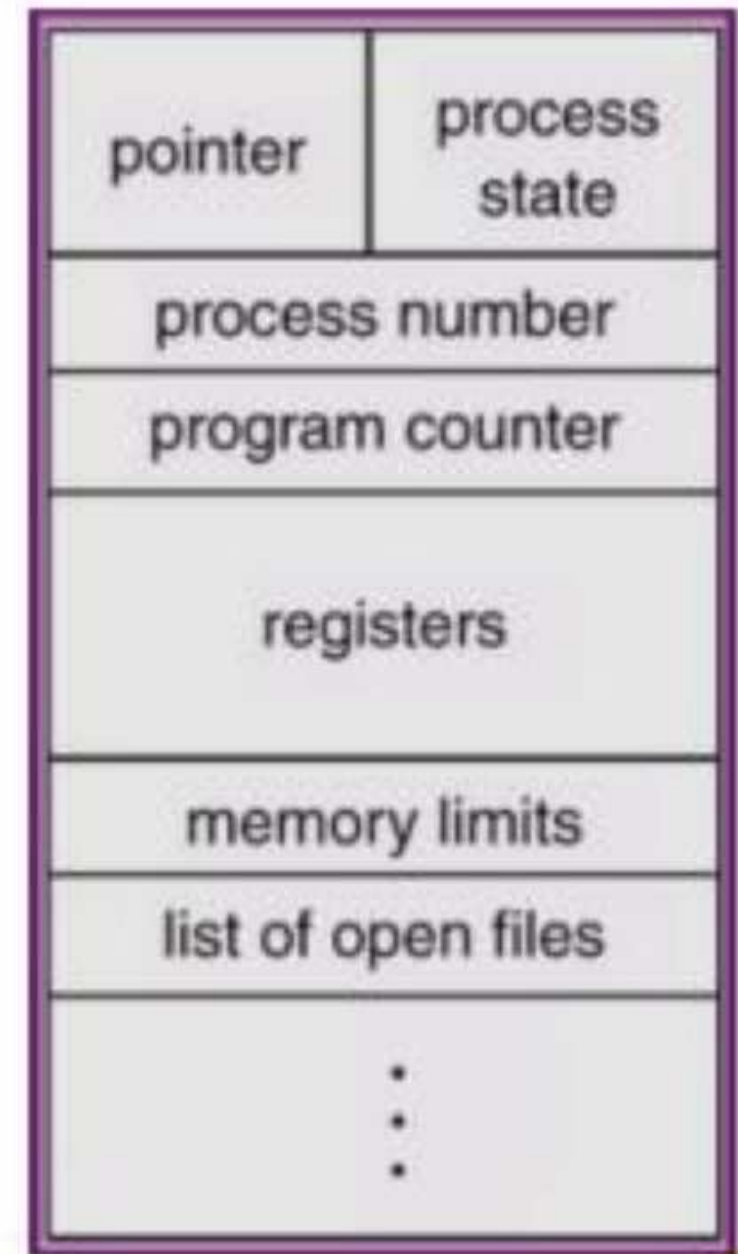


Process Control Block (PCB)

- A **process control block (PCB)** is a data structure used by computer operating systems to store all the information about a process.
- When a process is created (initialized or installed), the operating system creates a corresponding process control block.
- Process control block includes CPU scheduling, I/O resource management, file management information etc.
- If that process gets suspended, the contents of the registers are saved on a stack and the pointer to the particular stack frame is stored in the PCB.
- By this technique, the hardware state can be restored so that the process can be scheduled to run again.

pointer	process state
process number	
program counter	
registers	
memory limits	
list of open files	
⋮	

- Process control block includes CPU scheduling, I/O resource management, file management information etc.
- If that process gets suspended, the contents of the registers are saved on a stack and the pointer to the particular stack frame is stored in the PCB.
- By this technique, the hardware state can be restored so that the process can be scheduled to run again.



S.N.	Information & Description
1	Process State The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	Process privileges This is required to allow/disallow access to system resources.
3	Process ID Unique identification for each of the process in the operating system.
4	Pointer A pointer to parent process.
5	Program Counter Program Counter is a pointer to the address of the next instruction to be executed for this process.
6	CPU registers Various CPU registers where process need to be stored for execution for running state.
7	CPU Scheduling Information Process priority and other scheduling information which is required to schedule the process.
8	Memory management information This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	Accounting information This includes the amount of CPU used for process execution, time limits, execution ID etc.
10	IO status information This includes a list of I/O devices allocated to the process.

Process Scheduling Queues

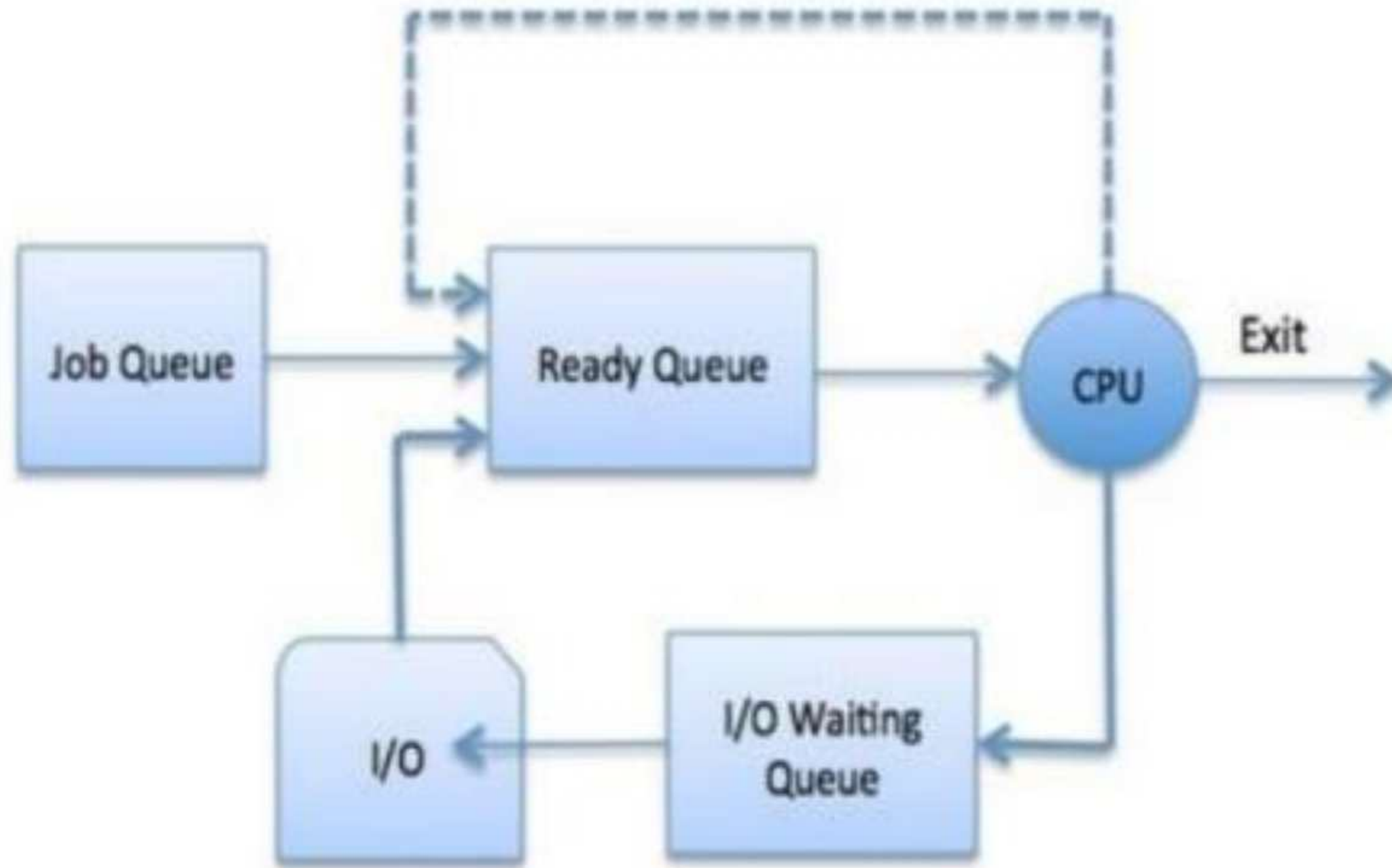
The OS maintains all PCBs in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following important process scheduling queues –

Job queue – This queue keeps all the processes in the system.

Ready queue – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.

Device queues – The processes which are blocked due to unavailability of an I/O device constitute this queue.



Schedulers

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run.

Schedulers are of three types –

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Long Term Scheduler

- It is also called a **job scheduler**.
- A long-term scheduler determines which programs are admitted to the system for processing.
- It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.
- The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound.
- It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

Short Term Scheduler

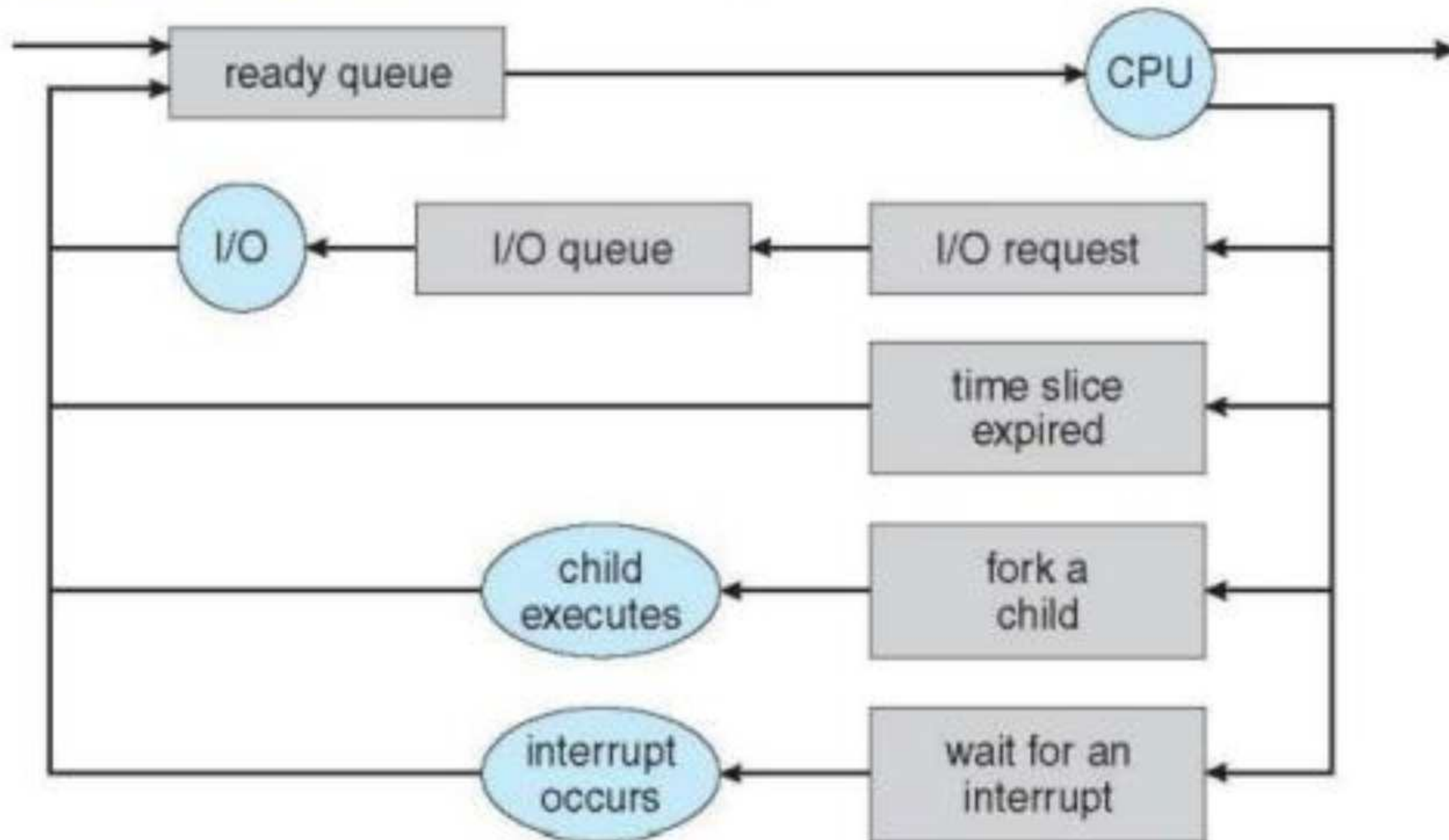
- It is also called as **CPU scheduler**. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.
- Short-term schedulers, also known as dispatchers, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

Medium Term Scheduler

- Medium-term scheduling is a part of **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.
- A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion.
- In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

Representation of Process Scheduling

- Queueing diagram represents queues, resources, flows



Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms. There are six popular process scheduling algorithms which we are going to discuss in this chapter –

➤ **First-Come, First-Served (FCFS) Scheduling**—In this Process which requests the cpu gets the cpu allocation first

➤ **Shortest-Job-Next (SJN) Scheduling**—Execution time should be selected for execution next

➤ **Priority Scheduling**—it selects the tasks to work as per the priority

➤ **Shortest Remaining Time**—The process will be allocated to the task, which is closest to its completion.

➤ **Round Robin(RR) Scheduling**—it works on principle, where each person gets an equal share of something in turn(change the position)

➤ **Multiple-Level Queues Scheduling**—it separate the queues. In this method processes are assigned to a queue based on a specific property

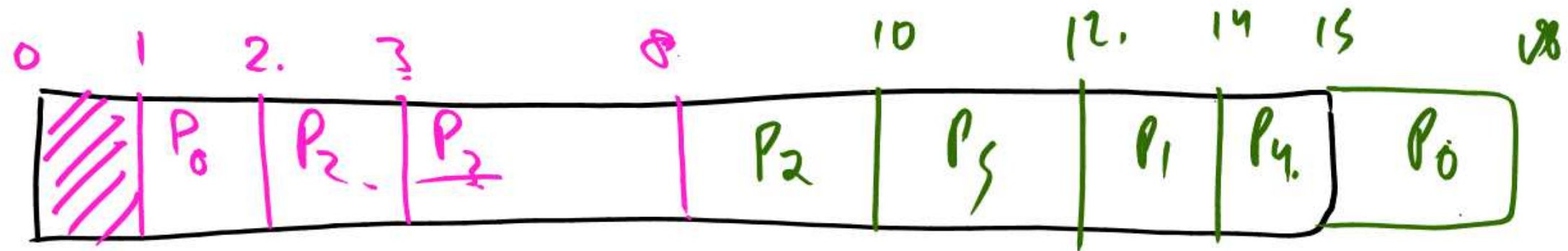
SSF, SJNF] - Theoretical Best time.
↳ minimum Average waiting time

→ Priority scheduling.

→ RR

	Priority AT	BT	Priority	CT	TAT	WT
P ₀	<u>1</u>	4 3	4	<u>18</u>	17	13
P ₁	2	2	5 ✓	14	12	10
P ₂	2	3 2	7	10	8	5
P ₃	3	<u>5</u>	8(H)	8	5	0
P ₄	3	1	5 ✓	15	12	11
P ₅	4	2	6 ✓	12	8	6

$$\frac{45}{6} \Rightarrow 7.5$$

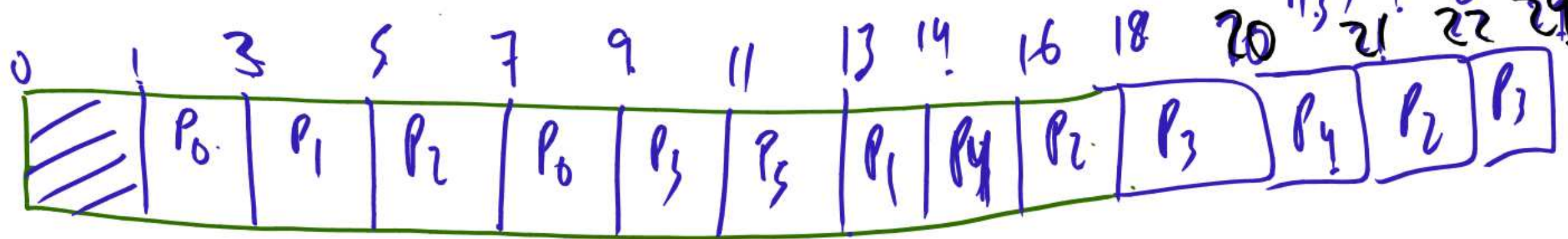


	AT	BT	CT	TAT	WT
P ₀	1	4 2	9		
P ₁	2	3 1	14		
P ₂	3	8 3	22		
P ₃	4	6 4	2		
P ₄	6	3	1		
P ₅	5	2			

Time slice = 2 sec

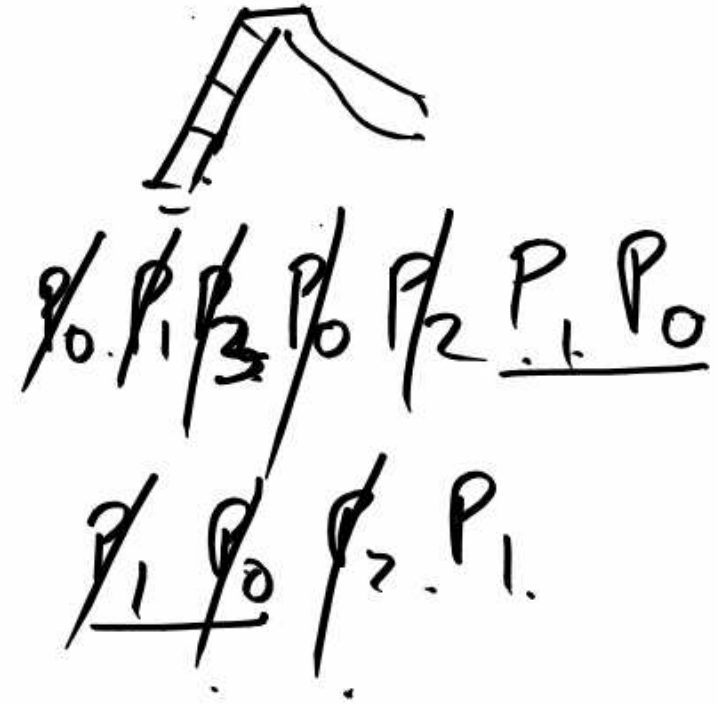
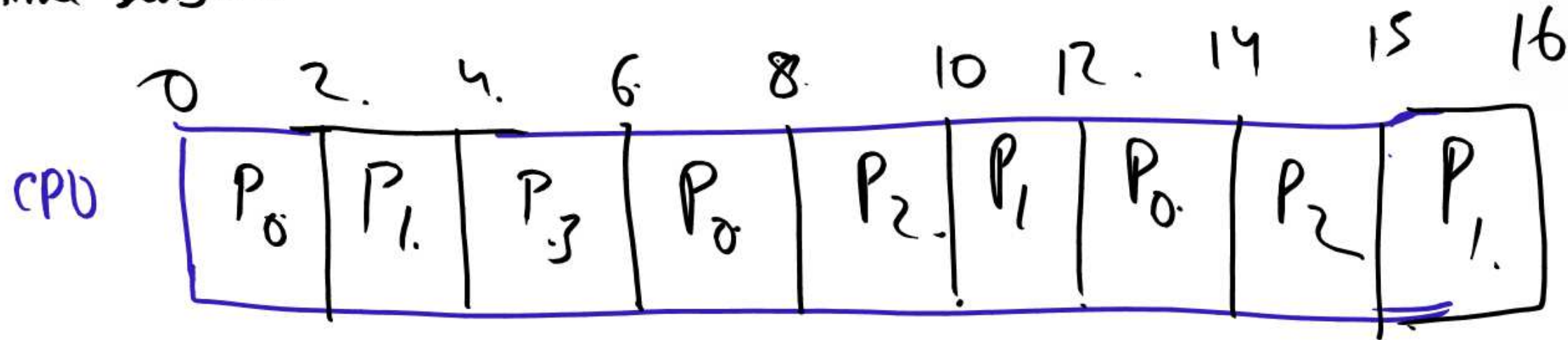
P₀ P₁ P₂ P₀ P₃ P₅ P₁ P₄ P₂

P₃ P₄ P₂ P₃



	AT	BT
P ₀	0	8.4.2.
P ₁	1	7.3.4
P ₂	3	3.1
P ₃	2	2

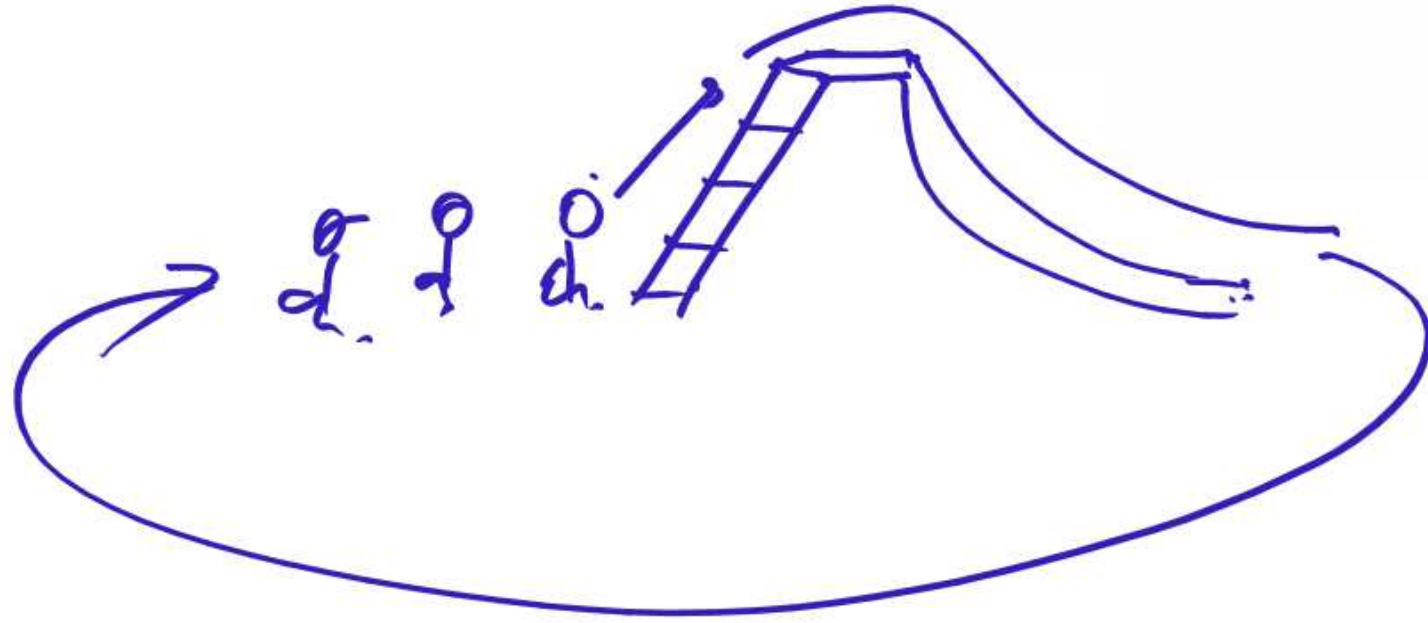
Time slot = 2



In priority scheduling process with low priority can go in starvation.

which can be handled by using ageing concept.

Round Robin. (RR) → time slice.
preemption



FCFS

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive, pre-emptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

- ☐ This is also known as **shortest job first**, or SJF
- ☐ This is a non-preemptive, pre-emptive scheduling algorithm.
- ☐ Best approach to minimize waiting time.
- ☐ Easy to implement in Batch systems where required CPU time is known in advance.
- ☐ Impossible to implement in interactive systems where required CPU time is not known.
- ☐ The processor should know in advance how much time process will take.

- Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a **quantum**.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted process.

Quantum = 3

Multiple-Level Queues Scheduling

- Multiple-level queues are not an independent scheduling algorithm. They make use of other existing algorithms to group and schedule jobs with common characteristics.
- Multiple queues are maintained for processes with common characteristics.
- Each queue can have its own scheduling algorithms.
- Priorities are assigned to each queue.
- For example, CPU-bound jobs can be scheduled in one queue and all I/O-bound jobs in another queue. The Process Scheduler then alternately selects jobs from each queue and assigns them to the CPU based on the algorithm assigned to the queue.

- A Thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers .
- It is also called as light weight process
- A thread is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack.
- It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

Threads

A thread is a lightweight process that runs concurrently with other threads within the same process. Threads share the same memory space and resources as the parent process.

Key Characteristics:

1. **Lightweight:** Threads are lighter than processes, requiring fewer resources.
2. **Shared memory:** Threads share the same memory space as the parent process.
3. **Concurrent execution:** Threads run concurrently, improving system performance.

Process vs. Thread

Process:

1. **Heavyweight:** Processes are heavier, requiring more resources.
2. **Separate memory space:** Each process has its own memory space.
3. **Independent execution:** Processes run independently.

Thread:

1. **Lightweight:** Threads are lighter, requiring fewer resources.
2. **Shared memory space:** Threads share the same memory space as the parent process.
3. **Concurrent execution:** Threads run concurrently.

Key differences:

1. **Memory:** Processes have separate memory spaces; threads share the same memory space.
2. **Resource usage:** Processes require more resources; threads require fewer resources.
3. **Communication:** Inter-process communication (IPC) is more complex; inter-thread communication is simpler.
4. **Creation:** Creating a process is more expensive; creating a thread is less expensive.

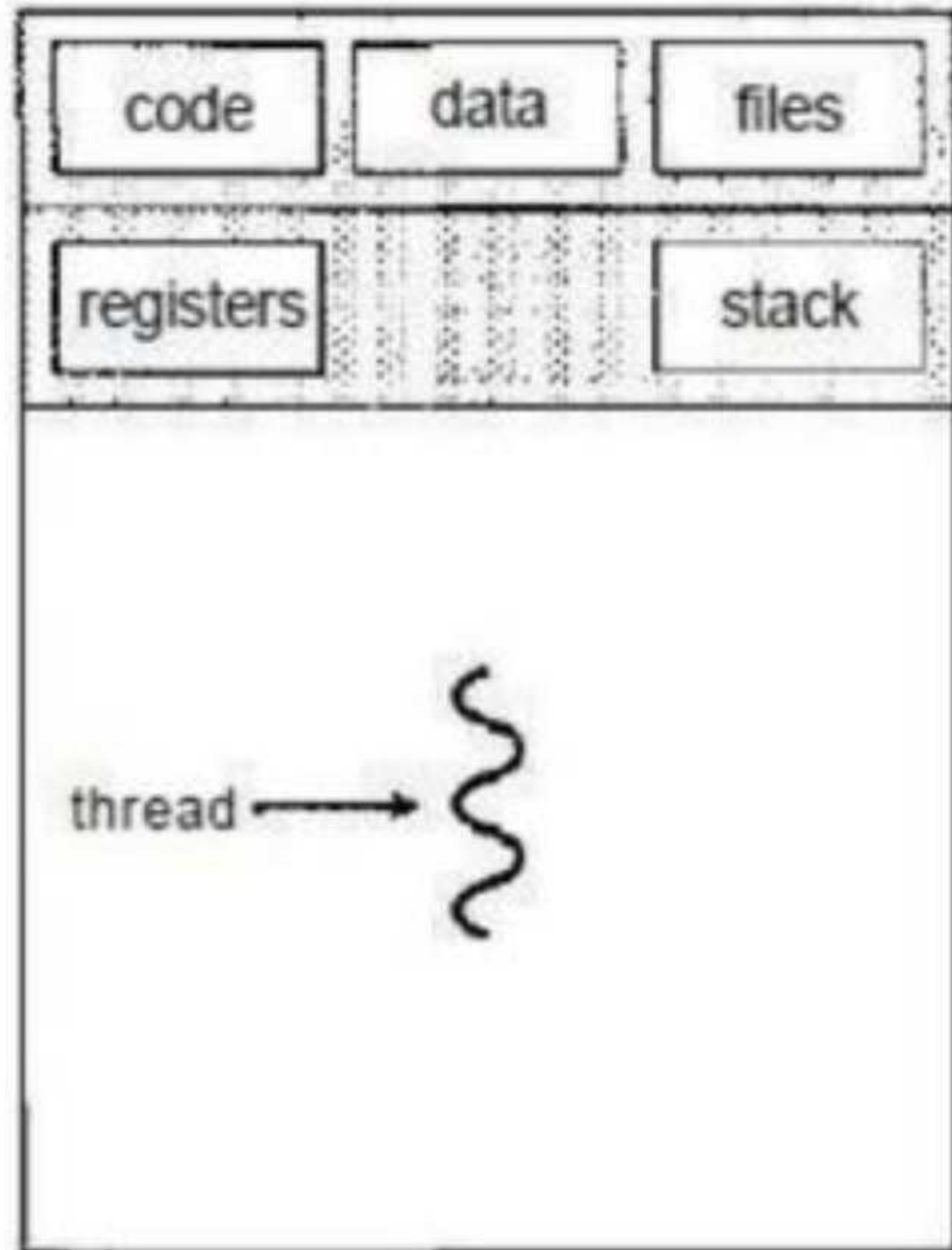
Difference between process and thread

Process

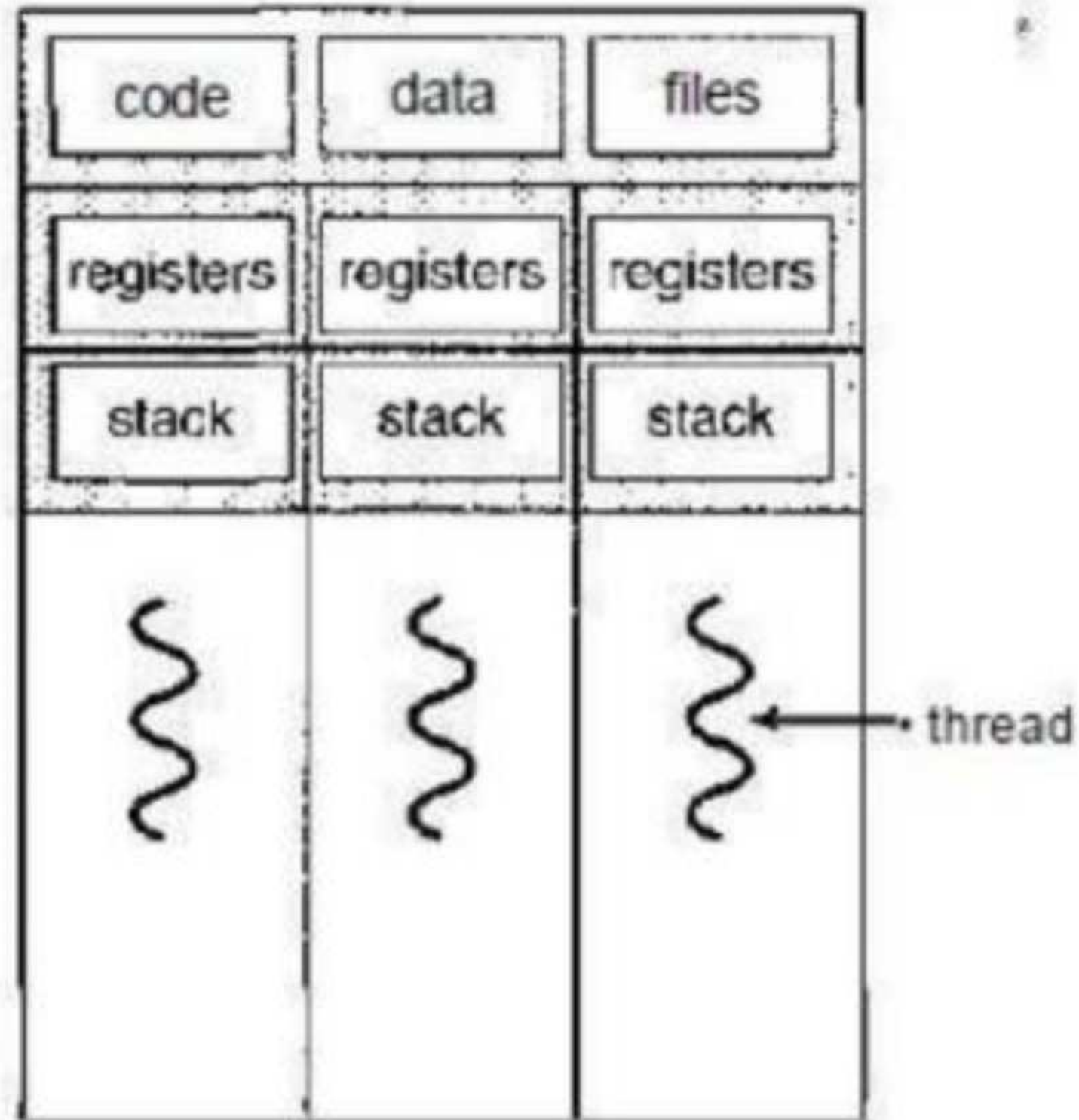
1. Process is heavy weight
2. It needs interaction with OS
3. It creates own memory of files
4. Multiple processes without using Threads use more resources
5. It is an independent

Thread

1. Light weight
2. Does not required with OS
3. it share same set of open files, diff processes
4. multiple threaded processes are fewer resources
5. one thread having read, write or change another thread



single-threaded process



multithreaded process

1. A Process Control Block(PCB) does not contain which of the following?

a) Code ✓

b) Stack ✓

c) Bootstrap program

d) Data ✓

2. The number of processes completed per unit time is known
as _____

- a) Output
- b) Throughput
- c) Efficiency
- d) Capacity

3. The state of a process is defined by _____

- a) the final activity of the process
- b) the activity just executed by the process
- c) the activity to next be executed by the process
- d) the current activity of the process

4. Which of the following is not the state of a process?

- a) New
- b) Old
- c) Waiting
- d) Running

new

Ready

Runny

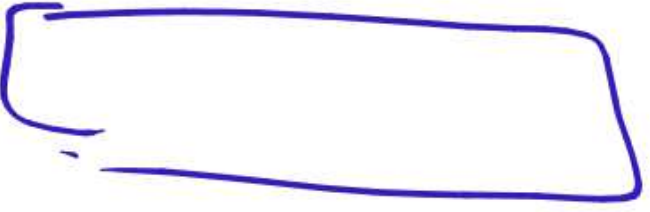
Block / Warky

Term...

5. What is a Process Control Block?

- a) Process type variable
- b) Data Structure
- c) A secondary storage section
- d) A Block in memory

6. The entry of all the PCBs of the current processes is in

-
- a) Process Register $\Rightarrow$ Data Register.
 - b) Program Counter $\rightarrow$ next instruction.
 - c) Process Table $\rightarrow$ 
 - d) Process Unit

7. What is the degree of multiprogramming?

- a) the number of processes executed per unit time. *(throughput)*
- b) the number of processes in the ready queue ✓
- c) the number of processes in the I/O queue
- d) the number of processes in memory ✓

8. A single thread of control allows the process to perform

-
- a) only one task at a time ✓
 - b) multiple tasks at a time ✗
 - c) only two tasks at a time
 - d) all of the mentioned

: A single thread of control allows the process to perform only one task at a time. In the case of multi-core, multiple threads can be run simultaneously and can perform multiple tasks at a time.

9. What is the objective of multiprogramming?

- a) Have a process running at all time
- b) Have multiple programs waiting in a queue ready to run
- c) To increase CPU utilization ✓✓
- d) None of the mentioned

Process Synchronization

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.
- Shared-memory solution to bounded-buffer problem (Chapter 4) allows at most $n - 1$ items in buffer at the same time. A solution, where all N buffers are used is not simple.
 - Suppose that we modify the producer-consumer code by adding a variable *counter*, initialized to 0 and incremented each time a new item is added to the buffer

Data integrity

Preamble



P₀

```
{ read(i)
  i++
  write(i) }
```

10
11

11

P₁

i = 10

```
{ read(i)
  i++
  write(i)
}
```

11

Q The following two functions P_1 and P_2 that share a variable B with an initial value of 2 execute concurrently.

$$I_{11} = C = 1$$

$$I_{12} = 2 = B$$

$$I_{21} = D = 4$$

$$I_{22} = B = 3$$

$P_1()$	$P_2()$
{	{
$I_{11} \quad C = B - 1;$	$I_{21} \quad D = 2 * B;$
$I_{12} \quad B = 2 * C;$	$I_{22} \quad B = D - 1;$
}	}

$$I_{11} = C = 1$$

$$I_{21} = D = 4$$

$$I_{22} = B = 3$$

$$I_{12} = B = 2$$

The number of distinct values that B can possibly take

Race Condition

- **Race condition:** The situation where several processes access – and manipulate shared data concurrently. The final value of the shared data depends upon which process finishes last.
- To prevent race conditions, concurrent processes must be **synchronized**.

General Structure of a process

- Initial Section: Where process is accessing private resources.
- Entry Section: Entry Section is that part of code where, each process request for permission to enter its critical section.
- Critical Section: Where process is access shared resources.
- Exit Section: It is the section where a process will exit from its critical section.
- Remainder Section: Remaining Code.

Data integrity

The Critical-Section Problem

- n processes all competing to use some shared data
- Each process has a code segment, called *critical section*, in which the shared data is accessed.
- Problem – ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section.

Solution to Critical-Section Problem

1. **Mutual Exclusion.** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress.** If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.

Bounded Waiting.

A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

- Assume that each process executes at a nonzero speed
- No assumption concerning relative speed of the n processes.

We generally have the following solutions to a Critical Section Problems:

1. Two Process Solution
 1. Using Boolean variable turn
 2. Using Boolean array flag
 3. Peterson's Solution
2. Operating System Solution
 1. Counting Semaphore
 2. Binary Semaphore
3. Hardware Solution
 1. Test and Set Lock
 2. Disable interrupt

P_0	P_1
<pre>while (1) { while (turn! = 0); Critical Section turn = 1; Remainder section }</pre>	<pre>while (1) { while (turn! = 1); Critical Section turn = 0; Remainder Section }</pre>

P_0

```
while (1)
```

```
{
```

```
    flag [0] = T;
```

```
    while (flag [1]);
```

```
    Critical Section
```

```
    flag [0] = F;
```

```
    Remainder section
```

```
}
```

 P_1

```
while (1)
```

```
{
```

```
    flag [1] = T;
```

```
    while (flag [0]);
```

```
    Critical Section
```

```
    flag [1] = F;
```

```
    Remainder Section
```

```
}
```

P_0	P_1
<pre> while (1) { flag [0] = T; turn = 1; while (turn == 1 && flag [1] == T); Critical Section flag [0] = F; Remainder section } </pre>	<pre> while (1) { flag [1] = T; turn = 0; while (turn == 0 && flag [0] == T); Critical Section flag [1] = F; Remainder Section } </pre>



2 mins Summary



TEACHING
WALLAH

Topic

Process synchronization
CPU scheduling

THANK YOU